Screenshots — Terraform Provisioners

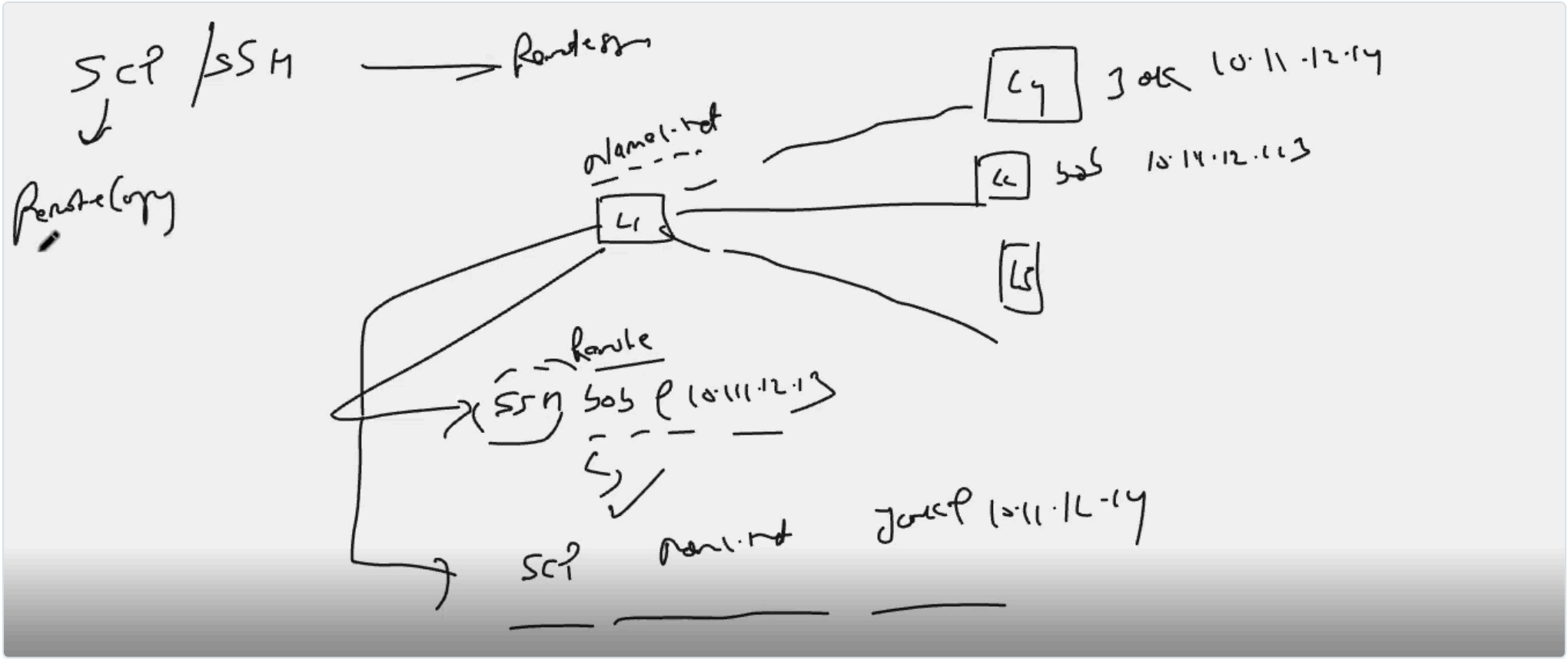
Lecture 4 · 25 shots, chronological · source stills from the session (2026-09-26 12:23 – 14:09)

01 / 25 Lecture definitions — Terraform provisioners (execute scripts/commands) and plugin cache (local storage for plugins) `12:23:01`

- terraform provisioners # Terraform Provisioners are tools used within a Terraform configuration to **execute scripts or** commands on a local or remote machine as part of the resource provisioning process.

- terraform plugin cache # Terraform plugin cache is like a **local storage that saves plugins** to make Terraform faster and more reliable.

Terraform provisioners are a built-in feature used to execute scripts, commands, or file transfers on local or remote machines during the creation or destruction of a resource. They act as an "imperative escape hatch" within Terraform’s otherwise declarative model, filling the gap when a cloud provider lacks a native mechanism to automatically bootstrap or configure a newly deployed resource. DEV Community +3


SCP / SSH
Remote copy
L4
L2
L1
L5
Router
SSH sob (10.111.12.13)
SCP
jump (10.11.12.14)

The Three Built-In Provisioners

HashiCorp includes three standard types of built-in provisioners to cover different infrastructure needs: HashiCorp Developer +2

- **`local-exec`** : Invokes a process or script locally on the machine running the Terraform CLI (e.g., your laptop or a CI/CD build agent).
- **`remote-exec`** : Connects to a newly created remote resource via SSH or WinRM to run designated shell commands directly inside that system.
- **`file`** : Copies files or directories securely from the host running Terraform over to the newly spawned remote resource. Scalr +3

hcl

```hcl
resource "aws_instance" "web_server" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  # Connection settings required by remote/file provisioners
  connection {
    type        = "ssh"
    user        = "ubuntu"
    private_key = file("~/.ssh/id_rsa")
    host        = self.public_ip
  }

  # 1. Copy a local configuration file to the new instance
  provisioner "file" {
    source      = "conf/nginx.conf"
    destination = "/tmp/nginx.conf"
  }
```

```
  # 1. Copy a local configuration file to the new instance
  provisioner "file" {
    source      = "conf/nginx.conf"
    destination = "/tmp/nginx.conf"
  }

  # 2. Run commands inside the instance to install Nginx
  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y nginx",
      "sudo mv /tmp/nginx.conf /etc/nginx/nginx.conf",
      "sudo systemctl restart nginx"
    ]
  }
}
```

```
resource "aws_instance" "web" {
  # ...

  provisioner "local-exec" {
    command = "echo The server's IP address is ${self.private_ip}"
  }
}
```

```
root@cka-w2:~/project1# ssh-keygen -f ram
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ram
Your public key has been saved in ram.pub
The key fingerprint is:
SHA256:RKkodocFPp1OTUmvTz2leWBuToCAV8HjHhqiH7qePqc root@cka-w2
The key's randomart image is:
+--[ED25519 256]--+
|     .o.+*+      |
|    ...o*=o      |
|     o=+o+.o o . |
|  o =++.o. = =   |
| . + o.+S.. O .  |
|  . . . .o + o   |
|   o             |
```

```
root@cka-w2:~/project1# ls
ram  ram.pub
```

```
root@cka-w2:~/project1# echo "I LOVE Terraform" > /tmp/redhat.txt
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# cat key.tf
resource "aws_key_pair" "my_blog_key" {

  key_name   = "my_tf_key"
  public_key = file("/root/project1/ram.pub")

}
```

```
root@cka-w2:~/project1# cat sg.tf
resource "aws_security_group" "cw_sg_ssh" {

  name = "cw-blog-3-sg-using-terraform"

  #Incoming traffic

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
root@cka-w2:~/project1# ls
key.tf  ram  ram.pub  sg.tf  terraform.log
root@cka-w2:~/project1# terraform init
```

```
root@cka-w2:~/project1# terraform init



- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.66.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

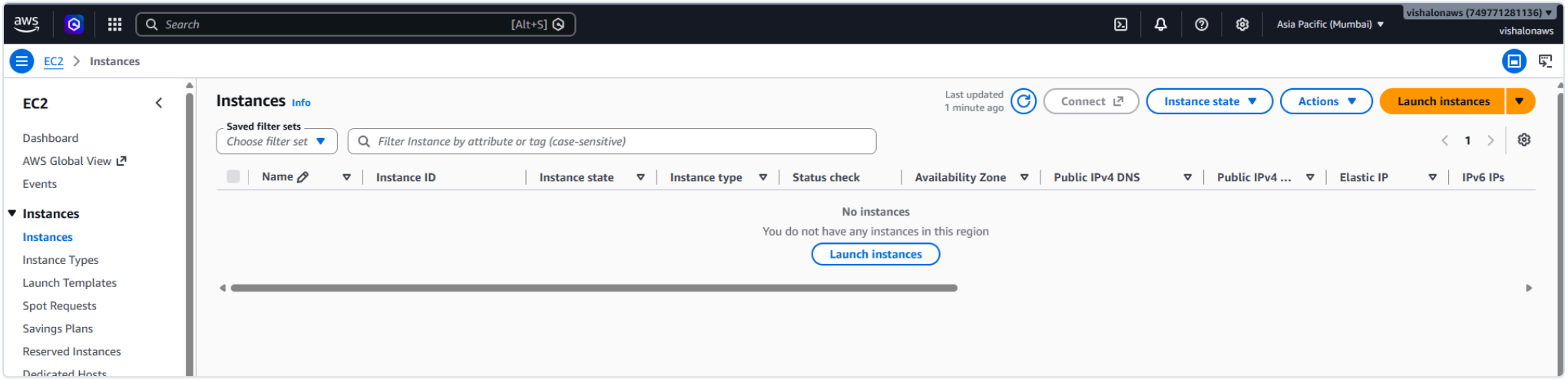
```
root@cka-w2:~/project1# terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

                              will be created
  + resource "aws_key_pair" "my_blog_key" {
      + arn             = (known after apply)
```

EC2 console (Mumbai) — “No instances”: only the key pair and SG exist so far



```
root@cka-w2:~/project1# terraform state list
aws_key_pair.my_blog_key
aws_security_group.cw_sg_ssh
root@cka-w2:~/project1#
```

```
resource "aws-instance" "web_server" {
   ami = "ami-066c4849e6b3a1e3d"
   instance_type = "t2.nano"
   key_name = "my_tf-key"
   security_groups = ["cw-blog-3-sg-using-terraform"]

}

# Connection settings required by remote/file provisioners
connection {

    type = "ssh"
    user = "ec2-user"
    private_key = "file("/root/project1/ram")
    host = self.public_ip # refers to the public IP of this particular instance
}
~
~
```

```
provisioner "file" {

source = "/tmp/redhat.txt"
destination = "/tmp/redhat1.txt"
}

#2 run commands inside the instance

provisioner "remote-exec" {

inline = [

   "sudo dnf update",
   "sudo dnf install -y httpd",
   "sudo echo hi this my basic web server > /var/www/html/index.html",
   "sudo systemctl enable --now  httpd"
      ]
}
```

```
# 3 Run command to local machine

provisioner "local-exec" {

command = "echo the servers IP address is ${self.private_ip} > /root/project1/m
y-node-ip.txt"

}
```

hcl

```hcl
resource "aws_instance" "web_server" {
  ami             = "ami-066c4849e6b3a1e3d"
  instance_type   = "t2.nano"
  key_name        = "my_tf-key"
  security_groups = ["cw-blog-3-sg-using-terraform"]

  # Connection settings required by remote/file provisioners
  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file("/root/project1/ram")
    host        = self.public_ip # refers to the public IP of this particular instance
  }

  # 1. Copy a local config file to the new instance
  provisioner "file" {
    source      = "/tmp/redhat.txt"
    destination = "/tmp/redhat1.txt"
  }

  # 2. Run commands inside the instance
  provisioner "remote-exec" {
    inline = [
      "sudo dnf update",
      "sudo dnf install -y httpd",
      "sudo echo hi this my basic web server > /var/www/html/index.html",
      "sudo systemctl enable --now  httpd"
    ]
  }

  # 3. Run command on local machine
  provisioner "local-exec" {
    command = "echo the servers IP address is ${self.private_ip} > /root/project1/my-node-ip.tx
  }
}
```

```
Terraform crashed! This is always indicative of a bug within Terraform.
Please report the crash with Terraform[1] so that we can fix this.

When reporting bugs, please include your terraform version, the stack trace
shown below, and any additional information which may help replicate the issue.
```

```
!!!!!!!!!!!!!!!!!!!!!!!! TERRAFORM CRASH !!!!!!!!!!!!!!!!!!!!!!!!

Terraform crashed! This is always indicative of a bug within Terraform.
Please report the crash with Terraform[1] so that we can fix this.

When reporting bugs, please include your terraform version, the stack trace
shown below, and any additional information which may help replicate the issue.

[1]: https://github.com/hashicorp/terraform/issues

!!!!!!!!!!!!!!!!!!!!!!!! TERRAFORM CRASH !!!!!!!!!!!!!!!!!!!!!!!!

panic: value is unknown
goroutine 137 [running]:
runtime/debug.Stack()
        runtime/debug/stack.go:26 +0x5e
github.com/hashicorp/terraform/internal/logging.PanicHandler()
        github.com/hashicorp/terraform/internal/logging/panic.go:84 +0x15c
panic({0x3ccdcc0?, 0x4a0e170?})
        runtime/panic.go:860 +0x13a
github.com/hashicorp/terraform/internal/terraform.(*Graph).walk.func1.1()
```

```
2 to add, 0 to change, 0 to destroy.



                                    Connecting to remote host via SSH...
                                      Host: 13.203.105.46
                                      User: ec2-user
                                      Password: false
                                      Private key: true
                                      Certificate: false
                                      SSH Agent: false
                                      Checking Host Key: false
                                      Target Platform: unix
                                    Connected!
                                    Waiting for process with pid 2515 to finish.
```

 Full apply log — instance created in 15s, `null_resource.configure_server` runs file → remote-exec; dnf waiting on pid 2515 14:09:07

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Creation complete after 15s [id=i-08009fcf02e77e2b4]
null_resource.configure_server: Creating...
null_resource.configure_server: Provisioning with 'file'...
null_resource.configure_server: Provisioning with 'remote-exec'...
null_resource.configure_server (remote-exec): Connecting to remote host via SSH...
null_resource.configure_server (remote-exec):   Host: 13.203.105.46
null_resource.configure_server (remote-exec):   User: ec2-user
null_resource.configure_server (remote-exec):   Password: false
null_resource.configure_server (remote-exec):   Private key: true
null_resource.configure_server (remote-exec):   Certificate: false
null_resource.configure_server (remote-exec):   SSH Agent: false
null_resource.configure_server (remote-exec):   Checking Host Key: false
null_resource.configure_server (remote-exec):   Target Platform: unix
null_resource.configure_server (remote-exec): Connected!
null_resource.configure_server (remote-exec): Waiting for process with pid 2515 to finish.
null_resource.configure_server: Still creating... [00m10s elapsed]
null_resource.configure_server: Still creating... [00m20s elapsed]
null_resource.configure_server: Still creating... [00m30s elapsed]
null_resource.configure_server: Still creating... [00m40s elapsed]
null_resource.configure_server: Still creating... [00m50s elapsed]
null_resource.configure_server: Still creating... [01m00s elapsed]
null_resource.configure_server: Still creating... [01m10s elapsed]
null_resource.configure_server: Still creating... [01m20s elapsed]
```